



Práctica 0: Entorno de prácticas. Vectores

Objetivos de la Práctica

1. Configurar el entorno de desarrollo en Visual Studio y el sistema de control de versiones con Git/GitHub.
2. Familiarizarse con la arquitectura de escenas (Scene y SceneManager) del proyecto base.
3. Implementar una clase matemática Vector3D propia con soporte de conversión (*wrapper*) para PhysX (PxVec3).
4. Aplicar los conceptos de álgebra vectorial (producto escalar, producto vectorial, normalización e interpolación) para resolver retos geométricos y visuales en la escena.

Introducción al entorno de prácticas. Repositorio de la asignatura

El código que vamos a usar a lo largo del curso va a partir de nuestro repositorio GitHub de la asignatura. Para acceder a él hay que visitar la siguiente dirección:

<https://github.com/UCM-237/SimulacionFisicaVideojuegos>

Tal y como se ha visto en clase, nuestro repositorio incluirá el código realizado por nosotros y la librería Physx de NVidia.

Actividad 1: Configuración del repositorio y del entorno local

- **Fork del repositorio:** Realizad un *fork* del repositorio oficial de la asignatura en vuestra cuenta personal de GitHub: <https://github.com/UCM-237/SimulacionFisicaVideojuegos>
- **Clonación local:** Abrid Visual Studio y seleccionad **Archivo** $\rightarrow$ **Clonar repositorio...**
 - En la **Ubicación del repositorio**, introducid la URL de vuestro *fork* (<https://github.com/><tu_usuario>/SimulacionFisicaVideojuegos).
 - Seleccionad una ruta local limpia en vuestro equipo y clonad el proyecto.
- **Comprobación:** Compilad y ejecutad la solución para verificar que la ventana de simulación abre correctamente.

Una vez realizada la copia del repositorio, vais a iniciar un proyecto dentro del entorno de desarrollo Visual Studio¹ a partir de dicha réplica. La idea va a ser que todo el código que vayáis desarrollando a lo largo de esta asignatura quede registrado en el repositorio del usuario. De esta forma podremos ver vuestra evolución e incluso podremos marcar entregas mediante etiquetas (tags) dentro del repositorio.

Actividad 2: Programación de la clase Vector3D

Cread las cabeceras/archivos necesarios para vuestra propia clase Vector3D. Esta clase será la herramienta matemática base para todas las prácticas de la asignatura. Debe incluir al menos:

- **Atributos públicos:** `float x, y, z;`
- **Constructores:** Por defecto $(0,0,0)$, por parámetros (x,y,z) y de conversión desde `physx::PxVec3`.
- **Módulo y Normalización:** `float magnitude() const` y `Vector3D normalize() const`.
- **Operaciones algebraicas:**
 - Producto escalar: `float dot(const Vector3D& v) const`
 - Producto vectorial: `Vector3D cross(const Vector3D& v) const`
 - Sobrecarga de operadores básicos: `=`, `+`, `-`, `*` (por un escalar) y `+=`.
- **Wrapper de conversión con PhysX:** Operador de conversión explícita/implícita `operator physx::PxVec3() const` para poder pasar variables de tipo Vector3D directamente a funciones de PhysX (`PxTransform`, `CreateShape`, etc.).

Actividad 3: Mi primera escena (P0S_Scene)

En el repositorio encontraréis la plantilla de la clase P0S_Scene (que hereda de `PhysicsEngine::Scene`).

1. **Creación de objetos:** En el método `init()`, pintad una esfera en el origen de coordenadas $(0,0,0)$ utilizando `CreateShape` y `RenderItem`. Podéis personalizar su color y escala libremente.
2. **Gestión de memoria:** En el método `cleanup()`, aseguraos de deregistrar y liberar de forma limpia los recursos creados invocando `.release()` sobre los punteros de `RenderItem` para evitar fugas de memoria.
3. **Navegación:** Verificad que podéis cambiar a vuestra escena en tiempo de ejecución pulsando la tecla '0' en el teclado.

Actividad 4: Retos de Geometría e Intuición Vectorial

Utilizando las funciones de vuestra recién creada clase Vector3D, resolved los siguientes retos dentro de P0S_Scene:

RETO A: SISTEMA DE REFERENCIA LOCAL (PRODUCTO VECTORIAL)

1. Dados dos vectores directores en el espacio:

¹ Nótese que dicho entorno está disponible con gratuitamente con licencia estudiante para todo el alumnado de la UCM.

- a. Eje local X: $\vec{u} = (3.0, 1.0, 0.0)$
 - b. Eje local Y: $\vec{v} = (0.0, 4.0, 0.0)$
2. **Ejercicio:** Calculad el vector perpendicular $\vec{w}$ mediante la función `cross()` que habéis programado.
3. Normalizad los tres vectores, escaladlos por 5.0 y representad mediante esferas el sistema de coordenadas resultante (Rojo = X, Verde = Y, Azul = Z).

Reflexión: ¿Hacia dónde apunta la esfera azul si invertís el producto vectorial $\vec{v} \times \vec{u}$?

RETO B: DETECCIÓN DE CAMPO DE VISIÓN (PRODUCTO ESCALAR)

1. Imaginad un enemigo situado en el origen (0,0,0) con dirección de visión $\vec{D} = (0.0, 0.0, 1.0)$
2. Posicionad esferas objetivo en: P_1 = (2.0, 0.0, 3.0), P_2 = (-4.0, 0.0, 1.0), P_3 = (0.0, 0.0, -5.0) y P_4 = (3.0, 0.0, 0.0).
3. **Ejercicio:** Utilizad el producto escalar `dot()` para clasificar visualmente el ángulo respecto a la visión del enemigo:
 - a. **VERDE:** si la esfera está **delante** del enemigo (ángulo < 90 grad. producto escalar positivo).
 - b. **ROJO:** si la esfera está **detrás** (ángulo > 90 grad, producto escalar negativo).
 - c. **AMARILLO:** si está justo en el plano perpendicular (90 grad., producto escalar nulo).

Reto C: Interpolación Lineal de Trayectoria (*Lerp*)

1. Dados dos puntos A = (-8.0, 1.0, -8.0) y B = (8.0, 8.0, 8.0):
2. **Ejercicio:** Usad la sobrecarga de operadores de vuestra `Vector3D` para generar 10 puntos intermedios a lo largo del segmento $P(t) = \vec{A} + t \cdot (\vec{B} - \vec{A})$ con $t \in [0, 1]$
3. Dibujad pequeñas esferas en cada punto P(t) para proyectar la trayectoria.